

Sistemi Informativi su Web aa 2025/2026

Progetto assegnato dal docente

1. Obiettivo

Il progetto assegnato dal docente consiste nella realizzazione di un piccolo sistema informativo su Web per la gestione di tornei di calcio amatoriale.

Il sistema deve essere implementato utilizzando:

- Spring Boot (backend)
- JPA / Hibernate (persistenza)
- PostgreSQL (o altro RDBMS)
- Thymeleaf (frontend)
- React (almeno per una parte del frontend)

L'obiettivo è progettare e implementare un sistema informativo completo, con particolare attenzione a:

- architettura a livelli
- gestione delle transazioni
- accesso efficiente ai dati
- separazione delle responsabilità

2. Scenario

Il sistema gestisce uno o più tornei di calcio amatoriale.

Per ogni torneo vengono registrate le squadre partecipanti, i giocatori, il calendario delle partite, i risultati e la classifica.

Il sistema distingue tra utenti generici, utenti registrati, e utenti amministratori. Questi ultimi possono accedere a funzionalità avanzate di gestione.

3. Entità principali

Il sistema deve includere almeno le seguenti entità.

Torneo

- nome
- anno
- descrizione

Aggiunta "Partecipazione"

- Squadra
- Torneo

Squadra

- nome

- anno di fondazione
- città

Relazioni (minime):

- una squadra partecipa a uno o più tornei
- una squadra ha più giocatori

Giocatore

- nome
- cognome
- data di nascita
- ruolo
- altezza

Relazioni (minime):

- ogni giocatore appartiene a una sola squadra

Partita

- data e ora
- luogo
- goalsHome
- goalsAway
- stato (es. SCHEDULED, PLAYED)

Relazioni (minime):

- una partita appartiene a un torneo
- una partita coinvolge due squadre
- una partita ha un arbitro

Arbitro

- nome
- cognome
- codice arbitrale

Relazioni (minime):

- un arbitro può dirigere più partite

Utente

- username
- password
- ruolo

4. Casi d'uso richiesti

4.1 Funzionalità pubbliche Completati!

- visualizzazione dell'elenco dei tornei
- visualizzazione del dettaglio di un torneo
- visualizzazione delle squadre partecipanti
- visualizzazione del calendario delle partite
- visualizzazione del dettaglio di una squadra (con giocatori)
- visualizzazione della classifica del torneo

4.2 Funzionalità utenti registrati

- visualizzazione commenti Fatto
- inserimento e modifica di un proprio commento ad una partita (un utente può modificare solo i propri commenti)

4.3 Funzionalità riservate all'amministratore Finite!

- creazione e modifica di un torneo
- inserimento e modifica di una squadra
- inserimento e modifica dei giocatori
- registrazione di una partita
- inserimento del risultato di una partita
- eliminazione di squadre e partite

5. Requisiti di sicurezza FATTO!

L'applicazione deve prevedere autenticazione e autorizzazione.

Ruoli

- USER: accesso alle funzionalità per utenti registrati
- ADMIN: accesso alle funzionalità di gestione

Requisiti minimi

- login con username e password
- protezione delle funzionalità amministrative

6. Architettura dell'applicazione

L'applicazione deve essere strutturata secondo una architettura a livelli.

Persistence layer

- repository JPA per l'accesso ai dati

Service layer

- implementa i casi d'uso
- coordina più repository
- contiene la logica di business
- gestisce le transazioni

Controller layer

- gestisce le richieste HTTP
- delega la logica ai service

7. Gestione delle transazioni

I metodi del service layer devono essere annotati con `@Transactional` quando modificano lo stato del sistema.

Occorre distinguere tra:

- operazioni di sola lettura
- operazioni di aggiornamento

La scelta del livello di isolamento deve essere coerente con il caso d'uso.

8. Prestazioni e accesso ai dati

Le operazioni di accesso ai dati devono essere progettate tenendo conto anche delle prestazioni.

8.1 Progettazione

Gli studenti devono prestare attenzione a:

- strategie di fetch
- presenza del problema di N+1 query

8.2 Analisi sperimentale (obbligatoria)

Per almeno un caso d'uso significativo, gli studenti devono:

- confrontare diverse strategie di accesso ai dati (ad esempio: LAZY, EAGER, join fetch, EntityGraph)
- eseguire una semplice sperimentazione
- misurare i tempi di esecuzione
- discutere i risultati

È sufficiente una analisi semplice ma chiara, purché evidenzii le differenze tra le strategie. L'obiettivo dell'analisi è quello di dimostrare la capacità di:

- progettare l'accesso ai dati in modo efficiente

- comprendere il comportamento di JPA / Hibernate
- motivare le scelte implementative

9. Frontend

L'applicazione deve includere almeno una parte realizzata in React. Il resto può essere realizzato con Thymeleaf.

10. Requisiti minimi

Il progetto deve includere:

- almeno 5–6 entità
- almeno 2 casi d'uso per ogni funzionalità (vedi Sezione 4). Tra questi, almeno un caso d'uso deve coinvolgere più entità
- autenticazione e autorizzazione

11. Criteri di valutazione

La valutazione terrà conto di:

- correttezza funzionale
- qualità del modello dati
- uso corretto di JPA / Hibernate
- qualità del service layer
- gestione delle transazioni
- gestione della sicurezza
- efficienza dell'accesso ai dati
- qualità del codice
- chiarezza dell'interfaccia utente
- validazione dei dati
- gestione degli errori

12. Bonus facoltativi

Saranno considerati elementi migliorativi:

- paginazione
- ricerca e filtri
- upload immagini
- autenticazione con OAuth
- deployment su una piattaforma cloud

13. Consegna

Inviare un messaggio di posta elettronica all'indirizzo **siw.roma3@gmail.com** entro le ore 18:00 del giorno precedente alla data dell'orale.

ATTENZIONE: non inviare il messaggio all'indirizzo di posta del docente.

L'oggetto del messaggio deve iniziare con la stringa [Giugno/Luglio 2026 PROGETTO DOCENTE] seguita da cognome e matricola dello studente. Esempio: lo studente Francesco De Rossi con matricola 123456 che si prenota per l'appello di giugno l'oggetto del messaggio deve essere: **[Giugno 2026 PROGETTO DOCENTE] De Rossi 123456**.

Nel corpo del messaggio riportare:

- indirizzo github (o altra piattaforma git) del progetto (con codice sorgente);
- eventuali malfunzionamenti noti, ma non risolti;
- considerazioni generali sull'esperienza.

Nota: Se lo studente non supera l'esame o rifiuta il voto, all'esame successivo dovrà portare un nuovo progetto assegnato dal docente circa 20 giorni prima della prova orale.

Progetto personale

1. Obiettivi

Progettare il proprio sistema, definendone casi d'uso, modello di dominio (con indicazioni utili alla progettazione dello strato di persistenza).

Implementare almeno 6 casi d'uso:

- almeno uno che richieda operazioni di inserimento dei dati di una entità
- almeno uno che richieda aggiornamento (update) dei dati di una entità
- almeno uno che richieda cancellazione di una (o più) entità
- almeno due che richiedano operazioni di lettura dei dati di una o più entità

NB: autenticazione e registrazione dell'utente non sono considerati casi d'uso.

Il sistema deve essere implementato utilizzando:

- Spring Boot (backend)
- JPA / Hibernate (persistenza)
- PostgreSQL (o altro RDBMS)
- Thymeleaf (frontend)
- React (almeno per una parte del frontend)

Il deploy su una piattaforma cloud (ad esempio, AWS, Azure, o Heroku) e l'autenticazione tramite OAuth saranno considerati un plus.

2. Consegna

Inviare un messaggio di posta elettronica all'indirizzo **siw.roma3@gmail.com** entro le ore 18:00 del giorno precedente alla data dell'orale.

ATTENZIONE: non inviare il messaggio all'indirizzo di posta del docente.

L'oggetto del messaggio deve iniziare con la stringa [Giugno/Luglio 2026 PROGETTO PERSONALE] seguita da cognome e matricola dello studente. Esempio: lo studente Francesco De Rossi con matricola 123456 che si prenota per l'appello di giugno l'oggetto del messaggio deve essere: **[Giugno 2026 PROGETTO PERSONALE] De Rossi 123456**.

Nel corpo del messaggio riportare:

- indirizzo github (o altra piattaforma git) del progetto (con codice sorgente);
- eventuali malfunzionamenti noti, ma non risolti.

Nota: se lo studente non supera l'esame o rifiuta il voto, all'appello successivo potrà portare lo stesso progetto personale.